HAFTA 3 • PAZAR

LLM Fine-Tuning

LoRA · RAG · LangChain

QLoRA · PEFT · Vektör Veritabanı · Retrieval-Augmented Generation · LangChain · Agents

09:00 — 17:00 · 7 Saat · 4 Python Dosyası · 900+ Satır Kod · Hafızlık Kurs Finali

Günün Planı — Hafta 3 Pazar (Kurs Finali)

09:00–09:30
Geçen Hafta Özeti & LLM Yol Haritası

09:30–10:45
LoRA & QLoRA: Parametre-Verimli Fine-Tuning

10:45–11:00
☕ Kısa Mola

11:00–12:00
Vektör Veritabanları & Embedding Arama

12:00–13:00
RAG: Retrieval-Augmented Generation

13:00–14:00
🍽 Öğle Arası

14:00–15:30
LangChain: Chain, Agent, Memory, Tool

15:30–16:30
LLM Değerlendirme & Güvenlik (RLHF/RLAIF)

16:30–17:00
🎓 Kurs Kapanışı & Proje Sunumu

🎯 **Gün Sonu: LoRA fine-tune çalışan model + RAG pipeline + LangChain agent + Kurs bitiş sertifikası**

LoRA — Low-Rank Adaptation: Az Parametre, Büyük Güç

Δ W = A · B · r << d · Sadece %1 Parametre · QLoRA: 4-bit Quantization

LoRA: W_0 + ΔW = W_0 + A·B ↔ W_0 ∈ ℝ^(d×k), A ∈ ℝ^(d×r), B ∈ ℝ^(r×k), r << min(d,k)

W_0 — Donmuş Orijinal Ağırlık

Ön-eğitimli modelin ağırlıkları tamamen dondurulur. Gradyan hesaplanmaz → GPU belleği tasarrufu. Eğitim boyunca değişmez.

A — İndir (Down-project)

Rastgele Gaussian ile başlatılır. (d×r) boyutlu küçük matris. r tipik: 4, 8, 16, 64. Bilgiyi düşük boyuta indirir.

B — Çıkar (Up-project)

Sıfırlarla başlatılır → başlangıçta ΔW=0 (kararlı). (r×k) boyutlu küçük matris. Düşük boyutlu temsili geri açar.

α — Ölçekleme Faktörü

ΔW = (α/r)·A·B. α genellikle r ile aynı değer ya da 2×r. Öğrenme hızını LoRA bileşenine göre ayarlar.

QLoRA — 4-bit Quantization + LoRA

- NF4 (Normal Float 4): Daha az bilgi kaybı
- Double Quantization: Kuantizasyon sabitlerini de kuantize et
- Paged Optimizers: GPU bellek taşması önleme
- 65B model → tek 48GB GPU'da fine-tune!
- Performans kaybı: Tam FT'ye kıyasla <1%

Yöntem	Param%	GPU (7B)	Performans
Tam FT	100%	~56 GB	Referans
LoRA r=8	~0.5%	~14 GB	≈ Tam FT
LoRA r=64	~4%	~18 GB	≈ Tam FT
QLoRA	~0.5%	~10 GB	≈−1% Tam

Hedef modüller: q_proj, v_proj (tipik) | k_proj, o_proj, gate/up/down (daha kapsamlı) | PEFT: get_peft_model() otomatik LoRA ekleme

LoRA Fine-Tuning — Tam Kod Akışı

PEFT · Hugging Face Trainer · SFTTrainer · LoRA Config

lora_finetuning.py

```
from peft import LoraConfig, get_peft_model, TaskType
from transformers import AutoModelForCausalLM, AutoTokenizer
from transformers import TrainingArguments, Trainer

# ── 1. Model yükle (yarı hassasiyet) ──────────────
model = AutoModelForCausalLM.from_pretrained(
    'meta-llama/Llama-2-7b-hf',
    torch_dtype=torch.float16,
    device_map='auto',          # Multi-GPU/CPU otomatik
    load_in_4bit=True,          # QLoRA için
)

# ── 2. LoRA yapılandırması ────────────────────────
lora_config = LoraConfig(
    r=16,               # rank (4,8,16,32,64)
    lora_alpha=32,      # ölçekleme: alpha/r
    target_modules=['q_proj','v_proj','k_proj','o_proj'],
    lora_dropout=0.1,   # LoRA katmanlarına dropout
    bias='none',        # bias güncellenmez
    task_type=TaskType.CAUSAL_LM,
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# trainable: 8.4M / 6.7B = 0.12%
```

r nasıl seçilir?

r=8: Hız öncelikliyse, küçük domain adaptasyonu
r=16-32: Genel dil görevi
r=64+: Tam FT'ye yakın, büyük veri

Hangi modüller hedeflenmeli?

Minimum: q_proj + v_proj (attention)
Kapsamlı: tüm attention + MLP layer
BERT: query + value

Öğrenme hızı ayarı

Tam FT: 1e-5 – 2e-5
LoRA: 1e-4 – 3e-4 (10× büyük olabilir)
Isınma adımları: %3-10 toplam

Birleştirme & Dağıtım

model.merge_and_unload() → LoRA ağırlıkları orijinale birleştir. Hızlı çıkarım için. GGUF/GGML'e dönüştür.

```
pip install peft trl transformers accelerate bitsandbytes datasets
trl: RLHF/SFT araçları
```

| bitsandbytes: QLoRA 4-bit quantization |

trainer = SFTTrainer(

Vektör Veritabanları & Semantik Arama

Embedding Modelleri · Cosine Similarity · FAISS · ChromaDB · Pinecone

Semantik Arama Sorunu

'Otomobil güvenliği' → anahtar kelime araması 'car safety' bulamaz. Vektör embedding'ler anlamı yakalar, kelime formu değil.
Geleneksel DB: SELECT * WHERE text LIKE '%..%' → Vektör DB: en_yakın_komşu(sorgu_vektörü, k=10)

Metin / Belge → Embedding Modeli (BERT, E5, BGE) → Yoğun Vektör (768-3072 boyut) → Vektör DB (FAISS/Chroma) → Benzer Belgeler (k-NN araması)

Benzerlik Metrikleri

Kosinüs Benzerliği (Cosine)

$\cos(\theta) = (A \cdot B)/(|A||B|) \in [-1, 1]$

Yön farkını ölçer, büyüklük değil. Metin için en yaygın. E5, BGE modelleri bunu optimize eder.

İç Çarpım (Dot Product)

$A \cdot B = \Sigma a_i b_i$

Hem yön hem büyüklüğü dikkate alır. OpenAI Ada embeddings varsayılanı. Normalize edilmişse = cosine.

L2 Mesafesi (Euclidean)

$d = \sqrt{\Sigma(a_i - b_i)^2} > 0$

Görüntü retrieval'da yaygın. Metinde kosinüsten az tercih edilir ama FAISS varsayılanı.

Popüler Vektör Veritabanları

FAISS (Meta)

CPU/GPU, bellekte çalışır. ANN araması. Prodüksiyon için en hızlı. Python kütüphanesi.

ChromaDB

Kalıcı disk depolama. LangChain entegrasyon yerel geliştirme ideali. Metadata filtreleme.

Pinecone

Yönetilen bulut servisi. Ölçeklenebilir, düşük gecikme. Metadata + hibrit arama.

Weaviate / Qdrant

Açık kaynak, prodüksiyon hazır. Vektör + anahtar kelime hibrid. Docker ile kolay kurulum.

Embedding modeli önerileri: BAAI/bge-m3 (çok dilli), intfloat/e5-large (İngilizce), text-embedding-3-large (OpenAI API) · pip install faiss-cpu chromadb sentence-transformers

RAG — Retrieval-Augmented Generation

Belge Parçalama · Vektör Arama · Bağlam Enjeksiyonu · Jenerasyon

RAG Akışı: Kullanıcı Sorusu → Embedding → Vektör Arama → İlgili Belgeler → Prompt'a Ekleme → LLM → Kaynaklı Cevap

Aşama 1: İndeksleme (Önceden)

1. **Belge Yükleme**
   PDF, HTML, TXT, CSV... Belge yükleyiciler
2. **Metin Parçalama**
   RecursiveCharacterSplitter. chunk_size=512, overlap=64
3. **Embedding Üretme**
   Her parça → embedding modeli → yoğun vektör
4. **Vektör DB'e Kaydet**
   ChromaDB / FAISS / Pinecone'a persist_directory ile

Aşama 2: Sorgulama (Gerçek Zamanlı)

1. **Soruyu Embed Et**
   Kullanıcı sorusu → aynı embedding modeli
2. **Benzer Parçaları Bul**
   k-NN araması: top-k=4 en yakın vektör
3. **Prompt Oluştur**
   Sistem + bağlam parçaları + kullanıcı sorusu
4. **LLM ile Cevapla**
   Grounded generation: 'Bağlama göre cevap ver'

rag_minimal.py

```
from langchain_community.vectorstores import Chroma
from langchain_huggingface import HuggingFaceEmbeddings
from langchain.chains import RetrievalQA
# İndeksleme
embedder = HuggingFaceEmbeddings(model_name='BAAI/bge-m3')
db = Chroma.from_documents(chunks, embedder, persist_directory='./chroma_db')
# Sorgulama
```

Advanced RAG: Hybrid Search (vektör+BM25) · Re-ranking · HyDE (Hypothetical Document Embedding) · Sentence-window retrieval · Self-RAG

LangChain — LLM Uygulama Çerçevesi

Chain · Agent · Memory · Tool · LCEL · Runnable

langchain_ornekler.py

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI  # veya HF modeli
from langchain_core.output_parsers import StrOutputParser

# ── LCEL Zinciri (LangChain Expression Language) ────
prompt = ChatPromptTemplate.from_template(
    'Şu konuyu Türkçe açıkla: {konu}')
llm   = ChatOpenAI(model='gpt-4o-mini')
chain = prompt | llm | StrOutputParser()
print(chain.invoke({'konu': 'Transformer attention'}))

# ── Conversation Memory ──────────────────────────
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain
memory = ConversationBufferMemory()
conv   = ConversationChain(llm=llm, memory=memory)
conv.predict(input='Merhaba!')      # t=1
conv.predict(input='Ne öğretirsin?')# t=2 — önceki bağlamı hatırlar

# ── Tool + Agent ─────────────────────────────────
from langchain.agents import AgentExecutor, create_react_agent
from langchain_community.tools import DuckDuckGoSearchRun
tools = [DuckDuckGoSearchRun()]
agent = create_react_agent(llm, tools, prompt=agent_prompt)
exec_ = AgentExecutor(agent=agent, tools=tools, verbose=True)
```

Chain (LCEL)

prompt | llm | parser zincirleme sözdizimi. Lazy evaluation, paralel dallar, streaming destekli.

Agent

LLM hangi aracı ne zaman kullanacağına karar verir. ReAct, OpenAI Functions, Custom Agent.

Memory

ConversationBuffer / Summary / Entity Memory. Geçmiş diyaloğu prompt'a enjekte eder.

Tool

DuckDuckGo, Python REPL, SQL, API, Vektör DB retriever. @tool dekoratörü ile özel araç.

LangGraph: DAG/graph tabanlı agent workflow · LangSmith: izleme/debug · pip install langchain langchain-community langchain-openai langchain-huggingface

LLM Değerlendirme & Güvenlik

Otomatik Metrikler · LLM-as-Judge · Red Teaming · Hallucination Tespiti

Otomatik Metrikler

- BLEU: n-gram örtüşmesi (çeviri, özetleme)
- ROUGE-L: En uzun ortak alt-dizi (özetleme)
- BERTScore: Embedding benzerliği (anlamsal)
- Perplexity: Dil modeli kalitesi (düşük=iyi)
- Exact Match / F1: QA görevleri için

LLM-as-Judge

- Güçlü LLM (GPT-4) zayıf modeli değerlendirir
- MT-Bench: 80 çok dönüşlü soru paketi
- Chatbot Arena: İnsan tercihi sıralama
- G-Eval: Bütünlük, tutarlılık, akıcılık, alaka
- Bias: Pozisyon, yineleme, kendi üretimini tercih

Güvenlik Katmanları

- Guardrails: NeMo Guardrails, Llama Guard
- Constitutional AI: Anayasa kuralları ile öz-eleştiri
- RLHF/RLAIF: İnsan/AI geri bildirim hizalama
- Red Teaming: Kötüye kullanım senaryoları testi
- Jailbreak: Prompt enjeksiyon tespiti

Hallucination Tespiti

- Self-Consistency: Aynı soruyu n kez sor, tutarsızlık var mı?
- Retrieval Check: Cevap bağlamda var mı?
- SelfCheckGPT: Örnekleme + çapraz doğrulama
- FActScore: Faktüel doğruluk atomik birimler
- RAG + kaynak atıfı: Her cümle için kaynak göster

TruthfulQA · ToxiGen · BBH · MMLU · HumanEval — standart benchmark suite · lm-evaluation-harness: tek komutla 60+ benchmark

LLM Dağıtım & Çıkarım Optimizasyonu

Quantization · vLLM · GGUF · TensorRT-LLM · Speculative Decoding

Quantization (Kuantizasyon)

FP32 → INT8/INT4 ile model boyutu küçültme. GPTQ (ağırlık), AWQ (aktivasyon farkında), BitsAndBytes (QLoRA). Kalite kaybı minimal.

```
# bitsandbytes INT8
from transformers import BitsAndBytesConfig
bnb_config = BitsAndBytesConfig(load_in_8bit=True)
model = AutoModel.from_pretrained(name,
quantization_config=bnb_config)
```

vLLM — Yüksek Verimli Servis

PagedAttention: KV cache verimliliği. Throughput 24× artış. OpenAI uyumlu API. Continuous batching ile düşük gecikme.

```
# vLLM sunucu başlatma
python -m vllm.entrypoints.openai.api_server
    --model meta-llama/Llama-2-7b-chat-hf
    --tensor-parallel-size 2  # Multi-GPU
```

GGUF / llama.cpp

CPU'da quantized model çalıştırma. Q4_K_M: 4-bit, kalite/hız dengesi. M1 Mac, Intel, ARM desteği. RAM → GPU offloading.

```
# llama.cpp Python binding
from llama_cpp import Llama
llm = Llama(model_path='model.Q4_K_M.gguf',
           n_gpu_layers=33, n_ctx=4096)
out = llm('Merhaba!', max_tokens=200)
```

Speculative Decoding

Küçük taslak model (draft) token önerir, büyük model batch'te doğrular. 2-3× hız artışı. Apple MLX, Google Medusa.

```
# Hugging Face speculative decoding
from transformers import pipeline
pipe = pipeline('text-generation',
    model=large_model,
    assistant_model=small_model  # taslak
```

Ollama: Yerel LLM çalıştırma arayüzü · LMStudio: GUI ile model yönetimi · TGI (Text Generation Inference): HF prodüksiyon sunucu

Bugünkü Uygulamalar — 4 Python Dosyası

pip install peft trl transformers faiss-cpu chromadb langchain sentence-transformers

01 LoRA Fine-Tuning: GPT-2 Talimat Modeli

`uygulama_01_lora_finetuning.py`

- LoRA sıfırdan: Low-Rank matris çarpımı NumPy demo
- PEFT + GPT-2 ile talimat fine-tuning (Alpaca formatı)
- r=4/8/16/32 ablasyonu — kalite vs parametre
- Eğitim öncesi/sonrası perplexity karşılaştırması
- model.merge_and_unload() ile birleştirme ve çıkarım

02 RAG Pipeline: Soru-Cevap Sistemi

`uygulama_02_rag_pipeline.py`

- PDF/metin belgesi → RecursiveCharacterSplitter parçalama
- HuggingFace embedding + FAISS / ChromaDB indeksleme
- Benzerlik araması: cosine distance görselleştirme
- Tam RAG pipeline: retrieve → re-rank → generate
- Hallucination kontrolü: cevap kaynak alıntısı

03 LangChain: Zincir, Agent ve Bellek

`uygulama_03_langchain.py`

- LCEL zinciri: prompt | llm | parser — 3 farklı görev
- ConversationChain: çok dönüşlü diyalog belleği
- ReAct Agent: web arama + Python REPL araçları
- Özel tool dekoratörü ile alan-özgü araç yazma
- LangGraph ile basit stateful workflow örneği

04 LLM Değerlendirme & Güvenlik

`uygulama_04_llm_degerlendirme.py`

- BLEU / ROUGE / BERTScore hesaplama ve görselleştirme
- LLM-as-Judge: GPT-4 ile otomatik yanıt puanlama
- Self-Consistency: hallucination tespiti
- Prompt enjeksiyon ve jailbreak tespiti simülasyonu
- Kapsamlı benchmark karşılaştırma tablosu

```
pip install peft trl transformers bitsandbytes faiss-cpu chromadb langchain langchain-community sentence-
```

Kurs Tamamlandı!

Son Haftanın Kazanımları:

- ✓ LoRA: r rank, α ölçekleme, hedef modüller tam anlaşıldı
- ✓ Vektör DB: FAISS, ChromaDB, embedding arama yapıldı
- ✓ LangChain: LCEL, Agent, Memory, Tool, ReAct döngüsü
- ✓ Dağıtım: vLLM, GGUF, quantization, speculative decoding
- ✓ QLoRA: 4-bit quantization ile büyük model fine-tuning
- ✓ RAG: İndeksleme → retrieval → generation tam pipeline
- ✓ LLM Eval: BLEU/ROUGE/BERTScore, LLM-as-Judge, hallucination
- ✓ Güvenlik: Red teaming, guardrails, prompt injection

Tebrikler! Klasik ML'den LLM'e tam yolculuğu tamamladınız. Sıradaki adım: Kendi projenizi geliştirin ve topluluğa katkıda bulunun!